

Backend Details SnipX

Jawad Manzoor

Main Application File

app.py - The Heart of the Backend

Purpose: This is the main server file that starts your application and handles all incoming requests from the frontend.

Key Concepts Used:

- Web Framework (Flask): Creates web server
- RESTful APIs: Structured way to communicate with frontend
- Middleware: Code that runs between request and response
- Error Handling: Managing when things go wrong

What it does:

1. Server Setup & Configuration

```
app = Flask(__name__)  
  
CORS(app, supports_credentials=True)
```

- Creates a Flask web server
- Enables CORS (Cross-Origin Resource Sharing) so frontend can talk to backend
- Sets up security configurations

2. Database Connection

```
client = MongoClient(os.getenv('MONGODB_URI'))  
  
db = client.snipx
```

- Connects to MongoDB database
- Creates database connection that all services use

3. API Endpoints (Routes)

The file defines multiple endpoints that frontend can call:

Authentication Routes:

- `/api/auth/login` - User login
- `/api/auth/register` - User registration
- `/api/auth/google/login` - Google OAuth login
- `/api/auth/facebook/login` - Facebook OAuth login

Video Processing Routes:

- `/api/upload` - Upload video files
- `/api/videos/<id>/process` - Process uploaded videos
- `/api/videos/<id>` - Get video information
- `/api/videos/<id>/download` - Download processed videos

Support System Routes:

- `/api/support/tickets` - Create/get support tickets
- `/api/chat` - Chat functionality

4. Security Middleware

```
def require_auth(f):

    def decorated(*args, **kwargs):

        auth_header = request.headers.get('Authorization')

        # Verify JWT token
```

- Checks if user is logged in before allowing access
- Uses JWT (JSON Web Tokens) for authentication

Dependencies File

requirements.txt - Python Package List

Purpose: Lists all Python libraries needed to run the backend.

Key Libraries Explained:

Web Framework

- `flask==3.0.0` - Main web framework
- `flask-cors==4.0.0` - Handles cross-origin requests

Database

- `pymongo==4.6.1` - MongoDB database driver
- `python-dotenv==1.0.0` - Loads environment variables

Video Processing

- `moviepy==1.0.3` - Video editing and manipulation
- `opencv-python==4.8.0.74` - Computer vision for video analysis
- `pydub==0.25.1` - Audio processing

AI/Machine Learning

- `tensorflow==2.19.0` - Machine learning framework
- `transformers==4.36.2` - Natural language processing
- `openai-whisper==20231117` - Speech-to-text conversion

- `torch==2.7.1` - PyTorch for deep learning

Utility Libraries

- `numpy==1.26.4` - Numerical computations
- `werkzeug==3.0.1` - WSGI utilities
- `gunicorn==21.2.0` - Production server

Utility Functions

utils.py - Helper Functions

Purpose: Contains reusable utility functions used across the application.

Concepts Used:

- File Handling: Working with files and directories
- MIME Type Detection: Identifying file types
- Security: File validation

Functions Explained:

1. File Type Detection

```
def get_mime_type(file_path):  
  
    mime = magic.Magic(mime=True)  
  
    return mime.from_file(file_path)
```

- Uses `python-magic` library to detect actual file type
- Prevents users from uploading malicious files disguised as videos

2. Video Validation

```
def is_valid_video(file_path):  
  
    mime_type = get_mime_type(file_path)  
  
    return mime_type.startswith('video/')
```

- Checks if uploaded file is actually a video
- Security measure to prevent harmful file uploads

3. Unique Filename Generation

```
def create_unique_filename(filename):  
  
    name, ext = os.path.splitext(secure_filename(filename))  
  
    counter = 1  
  
    while os.path.exists(f"uploads/{name}_{counter}{ext}"):   
  
        counter += 1  
  
    return f"{name}_{counter}{ext}"
```

- Prevents filename conflicts when multiple users upload files with same name
- Uses counter system to make filenames unique



Video Processing Engine

video_processor.py - Core Video Processing

Purpose: Handles actual video manipulation and processing tasks.

Concepts Used:

- Object-Oriented Programming (OOP): Class-based structure
- File I/O Operations: Reading and writing video files

- Audio Processing: Working with audio tracks
- Computer Vision: Image and video analysis

Class Structure:

VideoProcessor Class

```
class VideoProcessor:

    def __init__(self, filepath):

        self.filepath = filepath

        self.video = VideoFileClip(filepath)
```

Methods Explained:

1. Silence Cutting Algorithm

```
def cut_silence(self, threshold=-40, min_silence_len=500):
```

- Algorithm: Analyzes audio levels to detect silent parts
- Threshold: Audio level below which is considered silence (-40 dB)
- Process:
 1. Converts video to audio segments
 2. Identifies silent portions
 3. Removes silent segments
 4. Rejoins remaining parts

2. Thumbnail Generation

```
def generate_thumbnail(self, time=None):

    if time is None:

        time = self.video.duration / 2
```

```
frame = self.video.get_frame(time)
```

```
return frame
```

- Extracts a frame from middle of video
- Converts frame to image format
- Used for video previews

3. Audio Enhancement

```
def enhance_audio(self, boost_amount=1.5):
```

```
    audio = self.video.audio
```

```
    enhanced = audio.volumex(boost_amount)
```

```
    return enhanced
```

- Increases audio volume
- Applies audio filters
- Improves overall audio quality

Data Models (OOP Concepts)

models/ Folder - Data Structure Definitions

Purpose: Defines how data is structured and stored in the database.

Concepts Used:

- Object-Oriented Programming: Classes and objects
- Data Encapsulation: Keeping data and methods together

- Serialization: Converting objects to/from database format

models/user.py - User Data Model

Purpose: Defines user account structure and methods.

Class Structure:

```
class User:

    def __init__(self, email, password_hash, first_name=None, last_name=None):

        self.email = email

        self.password_hash = password_hash

        self.first_name = first_name

        self.last_name = last_name

        self.created_at = datetime.utcnow()

        self.videos = []

        self.settings = {}
```

Key Methods:

1. Object to Dictionary Conversion

```
def to_dict(self):

    return {

        "email": self.email,

        "password_hash": self.password_hash,

        # ... other fields

    }
```

- Converts Python object to dictionary
- Needed for storing in MongoDB database

2. Dictionary to Object Conversion

```
@staticmethod  
  
def from_dict(data):  
  
    user = User(  
  
        email=data["email"],  
  
        password_hash=data.get("password_hash"),  
  
        # ... other fields  
  
    )  
  
    return user
```

- Static method (doesn't need instance)
- Creates User object from database data
- Used when retrieving data from database

models/video.py - Video Data Model

Purpose: Represents video files and their processing status.

Attributes Explained:

- `user_id`: Who owns the video
- `filename`: Original file name
- `filepath`: Where file is stored on server
- `status`: Processing status (uploaded, processing, completed, failed)

- `processing_options`: What processing was requested
- `metadata`: Video information (duration, resolution, etc.)
- `outputs`: Results of processing (thumbnails, subtitles, etc.)

Processing States:

1. "uploaded" - File received, not processed yet
2. "processing" - AI is working on the video
3. "completed" - Processing finished successfully
4. "failed" - Something went wrong during processing

models/support_ticket.py - Support System Model

Purpose: Manages customer support requests.

Ticket Lifecycle:

1. "open" - New ticket created
2. "in_progress" - Admin is working on it
3. "resolved" - Issue fixed
4. "closed" - Ticket completed

Priority Levels:

- "low" - Minor issues
 - "medium" - Standard problems
 - "high" - Important issues
 - "urgent" - Critical problems
-

Business Logic Services

services/ Folder - Core Application Logic

Purpose: Contains the main business logic separated from web routing.

Concepts Used:

- Service Layer Pattern: Separating business logic from web layer
- Dependency Injection: Services receive database connection
- Single Responsibility Principle: Each service handles one area

services/auth_service.py - Authentication Service

Purpose: Handles user registration, login, and security.

Security Concepts Used:

- Password Hashing: Using bcrypt to securely store passwords
- JWT Tokens: JSON Web Tokens for session management
- Salt: Random data added to passwords before hashing

Key Methods Explained:

1. User Registration

```
def register_user(self, email, password, first_name=None, last_name=None):
```

```
    if self.users.find_one({"email": email}):
```

```
        raise ValueError("Email already registered")
```

```
    salt = bcrypt.gensalt()
```

```
    password_hash = bcrypt.hashpw(password.encode('utf-8'), salt)
```

Process:

1. Check if email already exists
2. Generate random salt
3. Hash password with salt
4. Store in database

Why Hashing?: Never store plain text passwords - if database is compromised, passwords are still protected.

2. User Login

```
def login_user(self, email, password):  
  
    user_doc = self.users.find_one({"email": email})  
  
    if not user_doc:  
  
        raise ValueError("Invalid email or password")  
  
  
    if not bcrypt.checkpw(password.encode('utf-8'), user_doc['password_hash']):  
  
        raise ValueError("Invalid email or password")
```

Process:

1. Find user by email
2. Compare provided password with stored hash
3. Generate JWT token if valid
4. Return token and user data

3. JWT Token Management

```
def generate_token(self, user_id):
```

```
payload = {  
    'user_id': str(user_id),  
    'exp': datetime.utcnow() + timedelta(days=1)  
}  
  
return jwt.encode(payload, self.secret_key, algorithm='HS256')
```

JWT Benefits:

- Stateless: Server doesn't need to store session data
- Secure: Cryptographically signed
- Expirable: Tokens automatically expire

services/video_service.py - Video Processing Service

Purpose: Manages video upload, processing, and AI operations.

AI/ML Concepts Used:

- Computer Vision: Analyzing video content
- Natural Language Processing: Generating summaries
- Speech Recognition: Converting audio to text
- Machine Learning Pipelines: Chaining AI operations

Key Processing Methods:

1. Video Upload & Validation

```
def save_video(self, file, user_id):  
  
    filename = secure_filename(file.filename)  
  
    filepath = os.path.join(self.upload_folder, filename)
```

```
if not self._is_valid_video(filepath):  
  
    os.remove(filepath)  
  
    raise ValueError("Invalid video file")
```

Security Measures:

- Sanitize filename to prevent directory traversal attacks
- Validate file type using MIME detection
- Remove invalid files immediately

2. AI-Powered Subtitle Generation

```
def _generate_subtitles(self, video, options):  
  
    # Extract audio  
  
    clip = VideoFileClip(video.filepath)  
  
    audio_path = f"{os.path.splitext(video.filepath)[0]}_audio.wav"  
  
    clip.audio.write_audiofile(audio_path)  
  
  
    # Use Whisper AI for transcription  
  
    import whisper  
  
    model = whisper.load_model("base")  
  
    result = model.transcribe(audio_data, language=whisper_lang)
```

AI Pipeline:

1. Audio Extraction: Separate audio from video
2. Speech Recognition: Convert speech to text using OpenAI Whisper

3. Language Detection: Identify spoken language
4. Timestamp Alignment: Match text with video timing
5. Format Generation: Create SRT and JSON subtitle files

3. Video Enhancement Processing

```
def _apply_video_enhancements(self, video, options):  
  
    brightness = options.get('brightness', 100) / 100.0  
  
    contrast = options.get('contrast', 100) / 100.0  
  
    def adjust_brightness_contrast(image):  
  
        img = image.astype(np.float32)  
  
        if brightness != 1.0:  
  
            img = img * brightness  
  
        if contrast != 1.0:  
  
            img = (img - 128) * contrast + 128  
  
        return np.clip(img, 0, 255).astype(np.uint8)
```

Image Processing Concepts:

- Pixel Manipulation: Modifying individual image pixels
- Brightness Adjustment: Multiplicative enhancement
- Contrast Enhancement: Adjusting difference between light and dark
- Clipping: Ensuring pixel values stay within valid range (0-255)

services/support_service.py - Customer Support Service

Purpose: Manages customer support tickets and communication.

Database Operations Used:

- CRUD Operations: Create, Read, Update, Delete
- Aggregation: Grouping and counting data
- Indexing: Fast data retrieval

Key Features:

1. Ticket Creation

```
def create_ticket(self, user_id, data):  
  
    ticket = SupportTicket(  
  
        user_id=user_id,  
  
        name=data.get('name'),  
  
        email=data.get('email'),  
  
        subject=data.get('subject'),  
  
        description=data.get('description'),  
  
        priority=data.get('priority', 'medium'),  
  
        ticket_type=data.get('type', 'bug')  
  
    )  
  
    result = self.tickets.insert_one(ticket.to_dict())
```

2. Statistics Generation

```
def get_ticket_stats(self):  
  
    pipeline = [  
  
        {  
  
            "$group": {  
  
                "_id": "$status",
```

```

        "count": {"$sum": 1}
    }
}

]

stats = list(self.tickets.aggregate(pipeline))

```

MongoDB Aggregation:

- Groups tickets by status
- Counts tickets in each group
- Returns statistics for admin dashboard

How Backend Connects to Frontend

API Communication Flow:

1. Frontend Request:

```

// Frontend sends request

const response = await fetch('/api/upload', {

  method: 'POST',

  headers: { 'Authorization': 'Bearer ' + token },

  body: formData

});

```

2.

3. Backend Processing:

```

# Backend receives and processes

```

```
@app.route('/api/upload', methods=['POST'])

@require_auth

def upload_video(user_id):

    file = request.files['video']

    video_id = video_service.save_video(file, user_id)

    return jsonify({'video_id': str(video_id)})
```

4.

5. Database Storage:

```
# Data saved to MongoDB

result = self.videos.insert_one(video.to_dict())
```

6.

7. Response to Frontend:

```
{

    "message": "Video uploaded successfully",

    "video_id": "64f8a1b2c3d4e5f6789012ab"

}
```

8.

Programming Concepts Summary

Object-Oriented Programming (OOP):

- Classes: User, Video, SupportTicket models
- Encapsulation: Data and methods bundled together
- Inheritance: Not heavily used but available

- Polymorphism: Different objects responding to same method calls

Data Structures & Algorithms (DSA):

- Arrays/Lists: Processing video segments
- Dictionaries/Hash Maps: JSON data structures
- Queues: Processing video jobs (implied)
- Search Algorithms: Finding users, videos in database
- Sorting: Ordering tickets by date, priority

Database Concepts:

- NoSQL: MongoDB document database
- CRUD Operations: Create, Read, Update, Delete
- Indexing: Fast data retrieval
- Aggregation: Data analysis and statistics

Security Concepts:

- Authentication: Verifying user identity
- Authorization: Checking user permissions
- Encryption: Password hashing, JWT signing
- Input Validation: Preventing malicious uploads

Software Architecture Patterns:

- MVC Pattern: Models, Views (frontend), Controllers (routes)
- Service Layer: Business logic separation
- Repository Pattern: Database access abstraction
- Middleware Pattern: Request/response processing

Concurrency & Performance:

- Asynchronous Processing: Video processing in background
- File Streaming: Handling large video uploads
- Caching: Storing processed results
- Load Balancing: Handling multiple users (production)